

Advanced Prompting Techniques



Chain-of-Thought Prompting

Key point: Make the AI think step by step instead of jumping to the final answer.

Why it helps: Developers can see not just the answer, but also how the AI got there. If the steps look wrong, you know the answer may also be wrong.

✗ Before

"Fix this bug: [paste code]."

✓ After

"Explain what this code does step by step, show me why it fails, and then give me the fixed version."



Tree-of-Thought Prompting

Key point: Make the AI explore several solution paths before choosing the best one.

Why it helps: In software, there are often many possible solutions. This technique gives you choices with pros and cons, not just one answer.

✗ Before

"How should I add login to my app?"

✓ After

"Suggest 3 ways to add login to a React + Node.js app (JWT, OAuth, NextAuth). Write pros and cons for each way, then recommend the best one for a small project."



Maieutic Prompting

Key point: Make the AI explain its own explanations for deeper understanding.

Why it helps: Some topics are complex. Developers may not just need the main answer, but also details about how and why it works.

✗ Before

"What is React hydration?"

✓ After

"Explain what React hydration is. Then explain: (1) what server-rendered HTML means, (2) how hydration attaches event listeners, (3) why hydration makes the app faster."



Complexity-Based Prompting

Key point: Generate multiple detailed answers and compare them to find the most reliable one.

Why it helps: Hard problems like algorithms often need detailed work. Longer answers usually mean the AI "thought" more carefully.

✗ Before

"How do I reverse a linked list in JavaScript?"

✓ After

"Write 3 different ways to reverse a linked list. Show step-by-step logic for each. Then pick the most reliable solution and explain why it is best."



Generated Knowledge Prompting

Key point: Make the AI collect key facts first, then use them to create the final answer.

Why it helps: If the AI first collects knowledge, the final answer is stronger and more accurate.

✗ Before

"Write an API for tasks."

✓ After

"First, list the most common features of a task manager API (create, update, delete, filter tasks). Then write an Express.js API that includes those features."



Least-to-Most Prompting

Key point: Solve easy parts first and build up to the harder ones.

Why it helps: This method reduces stress and keeps the work organised. It makes sure the AI builds the solution piece by piece.

✗ Before

"Build a chat app in React."

✓ After

"First, show me how to set up a React project with Vite."

"Now, show me how to add a chat input and message list."

"Finally, show me how to connect it to a WebSocket server for real-time messages."



Self-Refine Prompting

Key point: Let the AI review and improve its own answers in several passes.

Why it helps: The first answer is not always the best. By asking the AI to check itself, you get higher quality code or text.

✗ Before

"Write a function to sort an array."

✓ After

"Write a JavaScript function to sort an array. Then review the code for readability and performance. Rewrite it with improvements and add comments for clarity."



Directional-Stimulus Prompting

Key point: Guide the AI with clear rules, styles, or keywords for focused output.

Why it helps: Sometimes developers need answers in a very specific way — this method keeps the answer on track.

✗ Before

"Generate test cases."

✓ After

"Generate 5 unit test cases for the addUser function in UserService.js. Use Jest format. Include 3 success cases and 2 error cases."